What Makes Verdict Prediction Trivial? A Trace Ablation Study

Anonymous

May 15, 2026

Abstract

Execution trace supervised fine-tuning (trace SFT) has been proposed as a means of teaching language models to reason about program behavior. We investigate trace SFT for *verdict prediction*—binary pass/fail classification of test outcomes—and report a systematic negative result. Through ablation of a compact trace representation called VOTA (Verdict-Optimized Trace Abstraction), we show that (1) verdict tasks fundamentally require only assertion-point comparison data (actual vs. expected values), not behavioral reasoning from traces; (2) full execution traces are not merely unhelpful but actively harmful, and masking trace tokens from the loss (*label-only loss*) fails to repair the collapse, implicating context pollution rather than gradient dilution alone; and (3) VOTA's strong performance ($97.8\% \pm 1.3\%$) is a task-specific shortcut that does not transfer across models (DeepSeek-Coder: $+6.5$pp vs. CodeLlama: $+24.0$pp mean lift) or tasks (bug detection: $+3.5$pp, $p > 0.05$). These findings reframe VOTA as a precisely targeted answer-leakage mechanism rather than a reasoning scaffold, and characterize verdict prediction as an $O(1)$-information task for which trace SFT is the wrong paradigm.

1 Introduction

Execution traces—step-by-step records of program state during runtime—have attracted growing interest as training signal for code language models. DCSH [1] evaluated five trace formats on generative code tasks and found limited benefit. TracePile [2] scaled trace data to 2.6M samples but again evaluated only generation. TRACED [3] demonstrated that traces help encoder-based classification, but no prior work has examined decoder-based SFT for *verdict tasks*: binary classifications such as test pass/fail prediction.

Verdict tasks differ fundamentally from generation tasks in output entropy: a single bit versus an unbounded token sequence. This raises the question of whether trace representations designed for full execution simulation are appropriate for classification. To investigate, we introduced VOTA (Verdict-Optimized Trace Abstraction), a compact representation (80 tokens) that captures assertion-point state snapshots, exception summaries, and loop compression patterns.

Initial results were striking: VOTA-SFT achieved 99.16% accuracy on test verdict prediction with CodeLlama-7B, a $+27.65$ percentage point (pp) lift over the no-trace baseline (75.13%), while full execution trace SFT collapsed to 28.49%. This appeared to demonstrate that trace granularity matching is critical for classification tasks.

However, systematic ablation reveals a far simpler explanation. In this paper, we show that:

1. VOTA's entire performance gain derives from exposing **actual/expected value pairs** at assertion points—a direct answer key for the verdict. Removing these values reduces accuracy to 75.5%, statistically indistinguishable from the no-trace baseline.

2. Full trace collapse cannot be repaired by label-only loss masking. With trace tokens excluded from the loss, accuracy improves from 28.5% to only 42%, indicating that **trace content itself is harmful** at inference time, not just at training time.

3. VOTA's benefit is **model-specific** (CodeLlama: $+24.0$pp mean lift; DeepSeek-Coder: $+6.5$pp) and **task-specific** (test verdict: $+24.0$pp; bug detection: $+3.5$pp, not significant), ruling out a general trace design principle.

We argue that these negative results constitute a useful contribution to the trace SFT literature: they reveal that verdict prediction is an $O(1)$-information task whose information bottleneck is so narrow that any trace representation either (a) includes the answer key and trivializes the task, or (b) excludes it and provides no benefit. This characterization has implications for how the community evaluates trace-augmented training.

2 Related Work

Trace SFT for code. DCSH [1] introduced five execution trace formats (variable states, control flow, I/O pairs, step-by-step execution, and compressed summaries) and fine-tuned CodeLlama-7B and DeepSeek-Coder-6.7B on code generation tasks, finding modest and inconsistent improvements. TracePile [2] scaled to 2.6M trace-augmented samples and trained from scratch, reporting gains on HumanEval and MBPP but only evaluating generative benchmarks. Neither work examined classification tasks.

Traces for program understanding. TRACED [3] showed that execution traces improve encoder-based (CodeBERT, GraphCodeBERT) classification on tasks including clone detection and defect prediction. However, the encoder setting differs fundamentally from decoder SFT: encoders process traces as features, while decoders must generate traces as intermediate outputs before producing the final classification token.

Verdict prediction. Test outcome prediction has been studied as a downstream task in mutation testing and continuous integration optimization [4, 5]. These approaches typically use hand-engineered features (code coverage, historical pass rates) rather than trace-augmented LM fine-tuning.

3 Method

3.1 Task Definition

Given a code snippet c and an associated test case t, verdict prediction requires producing a binary label $y \in \{\texttt{pass}, \texttt{fail}\}$ indicating whether t passes when executed against c. The base rate in our dataset is 71.51% pass (majority class).

3.2 Trace Representations

We compare three trace conditions:

No trace. The model receives only c and t as input and directly predicts y. This serves as the baseline.

Full execution trace. The model receives c, t, and a complete step-by-step execution trace τ_{full} (4000 tokens on average), then predicts y. The trace records variable states, control flow decisions, and function calls at each execution step.

VOTA (Verdict-Optimized Trace Abstraction). The model receives c, t, and a compact trace abstraction τ_{VOTA} (80 tokens on average) containing three components:

- **Actual/expected value pairs**: For each assertion in t, the concrete values of the actual result and expected result at the assertion point.

Condition	Acc.	Lift	Δ vs Full
Full VOTA	99.16	+27.65pp	—
Actual/expected only	98.70	+23.60pp	-0.5pp
Exception summary only	76.20	+1.10pp	-23.0pp
Loop compression only	74.80	-0.30pp	-24.4pp
No-trace baseline	75.13	—	-24.0pp

Table 1: VOTA component ablation. Actual/expected value pairs account for nearly all of VOTA's lift. Exception summaries and loop compression contribute within noise margins.

- **Exception summary**: If execution raised an exception, the exception type and message.
- **Loop compression**: For loops, the iteration count and convergence pattern rather than per-iteration state.

3.3 Training Setup

All models are fine-tuned from CodeLlama-7B using LoRA (rank 16, $\alpha = 32$) for 3 epochs with learning rate 2×10^{-4}. We use standard causal language modeling loss unless otherwise noted. For the label-only loss variant, trace tokens are masked and loss is computed only on the final verdict token.

4 Experiments

We report seven experiments that systematically characterize why VOTA works and what this reveals about verdict task structure. All confidence intervals are 95% bootstrap CIs unless stated otherwise. We report mean $\pm$ std across 3 random seeds (42, 123, 456) where applicable.

4.1 Experiment 1: VOTA Component Ablation

To identify which VOTA components carry verdict-relevant signal, we train separate models with each component in isolation.

Table 1 reveals an extremely skewed contribution distribution. The actual/expected component alone recovers 98.70% accuracy—within 0.5pp of full VOTA—while exception summary (+1.1pp) and loop compression (-0.3pp) both fall within statistical noise of the no-trace baseline.

The 0.5pp gap between actual/expected-only and full VOTA likely reflects a small number of exception-based failure cases (e.g., `RuntimeError`) where no assertion is reached and hence no actual/expected pair exists, but the exception summary provides diagnostic signal.

This result directly implies that VOTA is not teaching the model to reason about execution behavior. Instead, it is exposing the *sufficient statistic* for the verdict: whether the actual output matches the expected output at each assertion point. The model learns a string comparison shortcut, not behavioral understanding.

4.2 Experiment 2: VOTA Without Actual/Expected Values

To confirm that VOTA's lift is entirely attributable to actual/expected exposure, we train a VOTA variant with these values removed, retaining only assertion structure (function names, assertion types, variable names), exception summaries, and loop compression.

Table 2 shows that without actual/expected values, VOTA's accuracy drops to 75.50%—a lift of merely 0.4pp over the no-trace baseline, well within the confidence interval overlap. The remaining VOTA components (assertion function names like `assertEqual`, variable names, exception types,

Condition	Acc.	Lift	95% CI
VOTA (full)	99.16	+27.65pp	[98.6, 99.7]
VOTA (no act/exp)	75.50	+0.40pp	[72.8, 78.2]
No-trace baseline	75.13	—	[72.4, 77.9]

Table 2: Removing actual/expected values from VOTA eliminates its entire performance advantage. The remaining trace structure provides no statistically significant lift.

Condition	Acc.	% Fail pred.	Status
Full trace (std. loss)	28.49	100%	collapse
Full trace (label-only)	42.00	62%	partial collapse
No trace (std. loss)	75.13	45%	normal
VOTA (std. loss)	99.16	28%	normal

Table 3: Label-only loss partially alleviates collapse but falls far short of recovery. The problem is not only gradient dilution—trace content itself corrupts the classification signal at inference time.

loop counts) do not carry sufficient information to determine whether a test passes or fails. Knowing *what kind of condition* is checked is categorically different from knowing *whether the condition holds*.

4.3 Experiment 3: Full Trace with Label-Only Loss

Full execution trace SFT collapses to 28.49% accuracy with 100% fail predictions. A natural hypothesis is gradient dilution: the loss on thousands of trace tokens overwhelms the single verdict token's gradient signal. If so, masking trace tokens from the loss should repair the collapse.

Table 3 shows that label-only loss improves accuracy from 28.49% to 42.00% and reduces fail-prediction bias from 100% to 62%. However, 42% remains far below the no-trace baseline (75.13%), indicating that gradient dilution is only one factor.

The deeper problem is what we term **context pollution**: trace content in the context window acts as an active noise source during inference. The model must attend to thousands of trace tokens whose distribution is drastically different from pretraining data (execution traces have a unique format the model has never encountered). This distributional shift corrupts the hidden states by the time the model reaches the verdict position. Label-only loss addresses the training-side gradient competition but cannot fix this inference-side corruption.

The persistent fail-prediction bias (62% fail predictions vs. 28.5% true fail rate) suggests that trace content systematically biases the model toward negative predictions, possibly because trace tokens correlate more strongly with error-related patterns in pretraining data.

4.4 Experiment 4: Trace Length Ablation

If context pollution scales with the number of irrelevant tokens, accuracy should decrease monotonically with trace length. We test this by truncating full traces to 256, 512, 1024, and 2048 tokens.

Table 4 confirms the monotonic decrease hypothesis. Critically, even 256 tokens of full trace—typically containing only initial variable assignments and imports—reduces accuracy by 17.1pp below the no-trace baseline. There is no "sweet spot" truncation length at which full traces become helpful.

This contrasts sharply with VOTA's 80 tokens, which provide a +27.65pp lift. The difference is not token count but *information density*: VOTA's tokens are the sufficient statistic for the verdict, while full trace tokens are overwhelmingly irrelevant to the pass/fail decision. Each additional trace token adds noise, and the cumulative effect drives the model toward collapse.

Trace length	Acc.	Δ vs no-trace
0 (no trace)	75.13	—
VOTA (80 tokens)	99.16	+27.65pp
256 tokens	58.00	−17.1pp
512 tokens	46.00	−29.1pp
1024 tokens	35.00	−40.1pp
2048 tokens	29.00	−46.1pp
Full (4000 tokens)	28.49	−46.6pp

Table 4: Accuracy vs. trace truncation length. Accuracy monotonically decreases with trace length. Even 256 tokens of full trace is harmful (−17.1pp vs. no trace), while VOTA's 80 tokens of *targeted* information yields +27.65pp. There is no beneficial truncation point for full traces.

Model	VOTA	No-trace	Full	VOTA lift
CodeLlama-7B	99.16	75.13	28.49	+24.0pp[†]
DeepSeek-6.7B	81.50	75.00	35.50	+6.5pp

Table 5: Cross-model comparison. VOTA lift varies by $4\times$ across models. Full trace collapse is consistent but less severe on DeepSeek-Coder. [†]Mean across 3 seeds.

4.5 Experiment 5: Cross-Model Validation

To test whether VOTA's effectiveness generalizes across model architectures, we replicate the three main conditions on DeepSeek-Coder-6.7B.

Table 5 reveals substantial cross-model inconsistency. VOTA's lift on DeepSeek-Coder-6.7B is only +6.5pp compared to +24.0pp (mean across seeds) on CodeLlama-7B—a $4\times$ difference. Given that seed variance on CodeLlama is $\pm$1.3pp, this gap far exceeds random variation.

This inconsistency is expected under the shortcut exploitation hypothesis: different models have different capacities for learning the actual/expected string comparison pattern. CodeLlama's tokenizer and pretraining distribution may be more conducive to this particular shortcut. If VOTA were teaching genuine behavioral reasoning, we would expect more consistent transfer across architecturally similar models of comparable scale.

Full trace collapse is replicated on DeepSeek-Coder (35.50%), confirming that context pollution is a robust phenomenon, though its severity is model-dependent.

4.6 Experiment 6: Cross-Task Generalization

We test VOTA on a second verdict task—bug detection on DebugBench (Python subset)—to evaluate whether the approach generalizes beyond test verdict prediction.

Table 6 shows that VOTA provides only +3.5pp lift on bug detection, which is not statistically significant. Full traces also collapse on bug detection (41.50%), though less severely than on test verdict prediction.

The failure to generalize is structurally predictable. Test verdict prediction has a clear "answer key"—the actual/expected comparison at each assertion point—that VOTA directly exposes. Bug detection has no analogous single-point sufficient statistic: determining whether code contains a bug requires understanding distributed semantic properties that no compact trace summary can reduce to a simple comparison. VOTA's actual/expected extraction logic has no equivalent in the bug detection setting.

This confirms that VOTA's effectiveness is task-specific shortcut exploitation, not a generalizable trace design principle. The implication is that there is no universal compact trace representation for verdict tasks: each task's information structure must be analyzed independently.

Task	VOTA	No-trace	Lift	Sig.
Test verdict	99.16	75.13	+27.65pp	yes
Bug detection	68.50	65.00	+3.50pp	no

Table 6: Cross-task comparison. VOTA's large lift on test verdict prediction does not transfer to bug detection ($p > 0.05$).

Seed	Accuracy	Error rate
42	99.16%	0.84%
123	96.50%	3.50%
456	97.80%	2.20%
Mean $\pm$ std	$97.8\% \pm 1.3\%$	$2.18\% \pm 1.33\%$

Table 7: Seed robustness for VOTA on CodeLlama-7B. The 2.66pp spread corresponds to a $4\times$ variation in error rate, suggesting instability in boundary cases.

4.7 Experiment 7: Seed Robustness and Calibration

Seed robustness. We train VOTA with three random seeds to quantify result stability.

Table 7 shows a 2.66pp spread across seeds, with error rates ranging from 0.84% to 3.50%—a $4\times$ variation. McNemar's test indicates that seeds 42 and 123 differ significantly ($p = 0.002$). While the main conclusion (VOTA $\gg$ no-trace) holds across all seeds, the instability in boundary cases is consistent with shortcut learning: the model's string comparison rule is reliable for clear-cut cases but fragile when actual/expected values are long, truncated, or formatted unusually.

Calibration. Table 8 shows that VOTA achieves low ECE (0.065) not through well-calibrated uncertainty estimates but through *task trivialization*: the model applies a near-deterministic rule ("actual == expected $\Rightarrow$ pass") with high confidence and is almost always correct. On its rare errors, VOTA is equally overconfident (>95% confidence), indicating that it has learned no genuine uncertainty awareness.

Full trace calibration is catastrophic: 88% average confidence at 28.5% accuracy, reflecting a collapsed model that confidently predicts fail regardless of the true label.

5 Discussion

5.1 Verdict Tasks as $O(1)$-Information Problems

Our ablation results converge on a structural characterization of verdict prediction: it is an $O(1)$-information task. The entire information required to determine a test verdict reduces to comparing two values (actual and expected) at each assertion point. This is fundamentally different from program generation or repair, where the model must synthesize unbounded output conditioned on rich contextual understanding.

This characterization explains three key findings simultaneously:

- **Why VOTA works**: It happens to include the $O(1)$ sufficient statistic.
- **Why full traces fail**: They embed the $O(1)$ signal in $O(n)$ noise, where n is the trace length. The signal-to-noise ratio decreases monotonically with n.
- **Why VOTA doesn't generalize**: Other verdict tasks (e.g., bug detection) may not have an $O(1)$ sufficient statistic, or it may not be extractable from execution traces.

Condition	ECE	Avg. conf.	Interpretation
VOTA	0.065	0.97	trivially calibrated
No trace	0.120	0.72	underconf. on fail
Full trace	0.580	0.88	catastrophic miscal.

Table 8: Expected calibration error (ECE). VOTA's low ECE is a byproduct of task trivialization (high confidence + high accuracy on a shortcut rule), not genuine uncertainty modeling. Full trace shows catastrophic miscalibration: 88% average confidence at 28.5% accuracy.

5.2 Context Pollution vs. Gradient Dilution

Prior work has attributed full-trace collapse primarily to gradient dilution [1]. Our label-only loss experiment (Table 3) demonstrates that this account is incomplete. Removing trace tokens from the loss partially alleviates collapse (28.5% $\rightarrow$ 42%) but falls far short of recovery (75.1% baseline), establishing that trace *content*—not just its gradient contribution—actively harms classification.

We propose **context pollution** as the complementary mechanism: trace tokens, whose distribution diverges sharply from pretraining data, corrupt the model's hidden representations through the attention mechanism. This is an inference-time phenomenon that no training-side intervention (loss masking, gradient scaling) can fully address while the trace remains in the context window.

5.3 Implications for Trace SFT Evaluation

Our results suggest several cautions for the trace SFT research community:

1. **Ablate trace components before claiming reasoning.** Strong performance with trace augmentation does not imply the model has learned to reason from traces. It may have learned a task-specific shortcut that happens to be embedded in the trace representation.

2. **Test cross-model and cross-task transfer.** A trace design that works on one model–task pair may exploit model-specific or task-specific properties rather than encoding general reasoning capabilities.

3. **Characterize task information structure.** Before designing a trace representation, analyze what information is actually needed to solve the task. For $O(1)$-information tasks, trace SFT may be the wrong paradigm entirely—direct feature extraction may be more appropriate.

5.4 Limitations

Our study has several limitations. First, we evaluate on a single dataset for the primary task (test verdict prediction) and a single secondary task (bug detection). Broader evaluation across verdict-type tasks would strengthen the generalization claims. Second, we test only 7B-scale models; larger models might extract useful signal from full traces. Third, our VOTA design is specific to test verdict prediction; the ablation methodology, rather than the specific representation, is our transferable contribution. Finally, seed robustness is evaluated with only three seeds; more seeds would provide tighter variance estimates.

6 Conclusion

We presented a systematic ablation study of trace SFT for verdict prediction tasks. Our central finding is negative: VOTA's strong performance on test verdict prediction is entirely attributable to actual/expected value pair exposure—a task-specific shortcut, not behavioral reasoning. Full execution traces are actively harmful through a context pollution mechanism that label-only loss cannot repair. These effects are model-specific and task-specific, ruling out general trace design principles.

We characterize verdict prediction as an $O(1)$-information task and argue that this structural property makes it fundamentally unsuitable for trace-based reasoning augmentation. For the trace SFT community, our results provide a cautionary case study: ablation, not just accuracy, is necessary to distinguish genuine reasoning from shortcut exploitation.

References

[1] Anonymous. Execution trace formats for code generation: A comparative study. In *Findings of EMNLP*, 2025.

[2] Anonymous. TracePile: Scaling execution trace data for code language models. In *NeurIPS*, 2025.

[3] Anonymous. TRACED: Execution-aware pre-training for source code. In *ICSE*, 2024.

[4] Anonymous. Predictive mutation testing with language models. In *ISSTA*, 2023.

[5] Anonymous. Optimizing CI pipelines with test outcome prediction. In *ASE*, 2024.